# Natural Language Processing

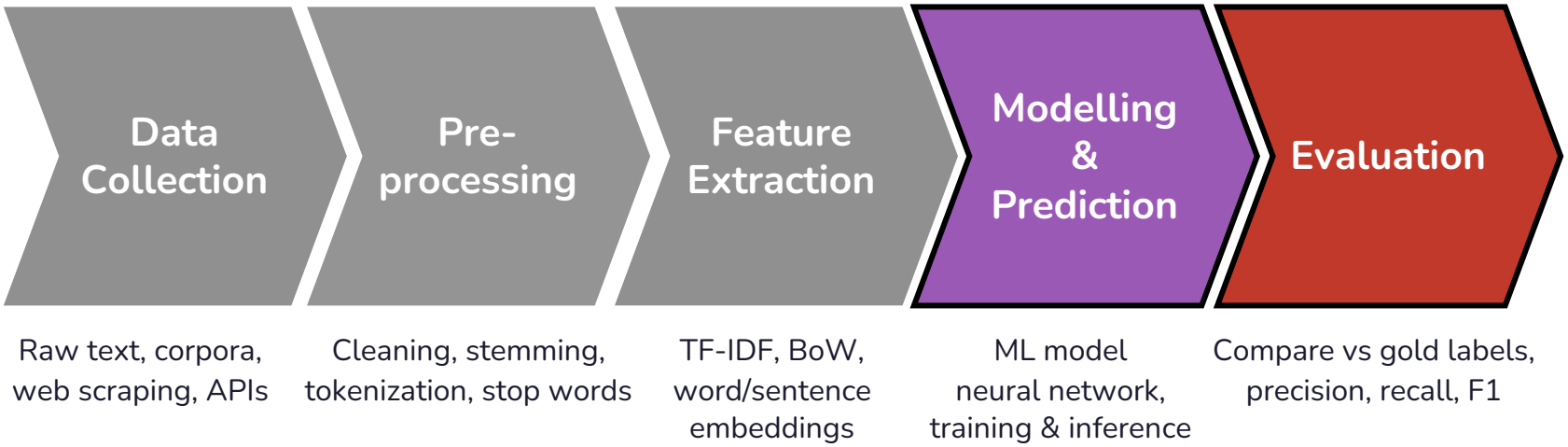
## Session 05 – Text Classification

Prof. Dr. Richard Sieg
SoSe 2026

# Schedule

| Date | Weekday | CW | Room | Topic |
|---|---|---|---|---|
| 16.04. | Thursday | 16 | 149 | Introduction: The World of NLP |
| 23.04. | Thursday | 17 | 149 | Text Processing Fundamentals |
| 30.04. | Thursday | 18 | 149 | Linguistic Annotations & Pattern Matching |
| 05.05. | Tuesday | 19 | 154 | Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05. |
| 07.05. | Thursday | 19 | 149 | Text Classification |
| 28.05. | Thursday | 22 | 149 | Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani |
| 01.06. | Monday | 23 | 390 | 📝 **Exams — Master DS Students** |
| 02.06. | Tuesday | 23 | 154 | BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06. |
| 11.06. | Thursday | 24 | 149 | The BERT Ecosystem |
| 18.06. | Thursday | 25 | 149 | Introduction to LLMs |
| 25.06. | Thursday | 26 | 149 | Working with LLMs |
| 02.07. | Thursday | 27 | 149 | Ethics, Limitations & Exam Preparation |
| 09.07. | Thursday | 28 | 149 | 📝 Exams (afternoon) — Bachelor DIS Students |
| 10.07. | Friday | 28 | 149 | 📝 Exams (afternoon) — Bachelor DIS Students |

# Recall: The typical NLP pipeline



# Agenda

01. Naïve Bayes

02. Perceptron

03. Logistic Regression

04. Other Classifiers

05. Evaluation

# What is Text Classification?

- ***Text classification:*** automatically assigning a predefined label to a text
- ***Input:*** a document d (email, review, tweet, article, lyric, …)
- ***Output:*** a class c from a fixed set $C = \{c_1, c_2, \ldots, c_j\}$
- ***Formally:*** Learn a function $F: D \rightarrow C$ that maps documents to classes
- This is a supervised task: we learn from labeled examples.

I got my mind on my money and my money on my mind → `Hip-Hop`

# The Core Task of NLP

**Knowledge Discovery**

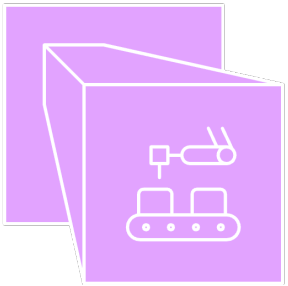
extraction of insights and detection of patterns within large corpora

**Enabling Technology**

Core component in higher-level NLP applications

**Information Management**

managing and navigating vast quantities of text



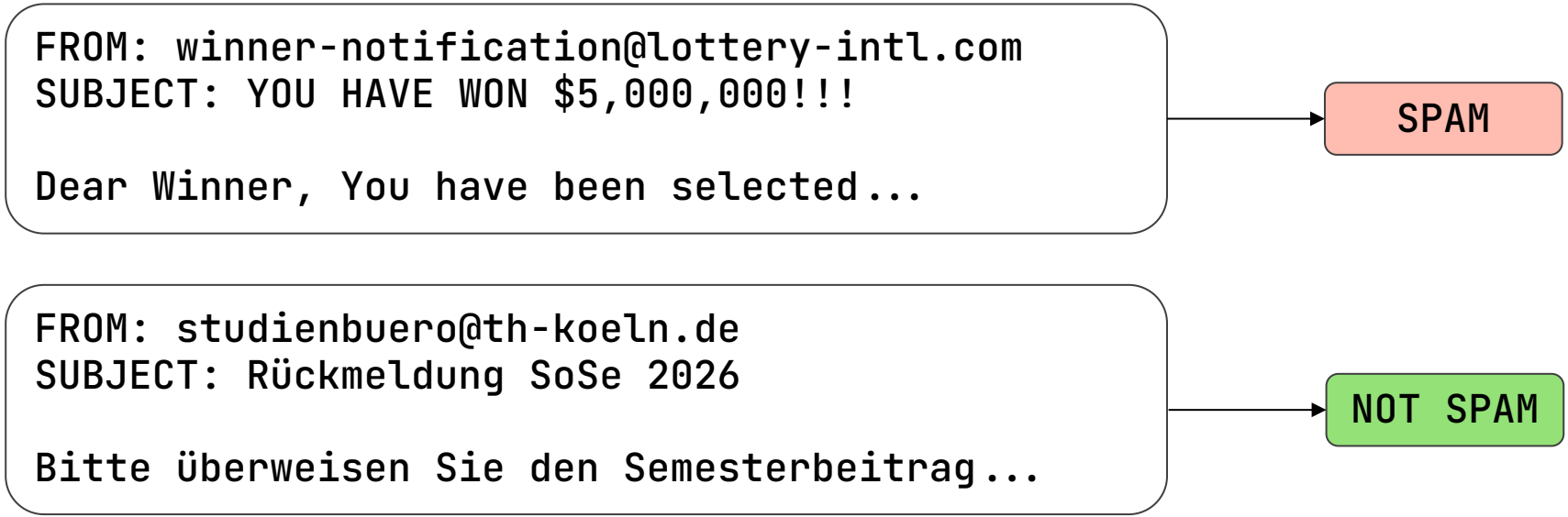
**Automation & Efficiency**

Automates processes for high efficiency, consistency and scalability.

- Classify Emails
- Voice Assistant (detect intent)
- Search Engines
- Content Moderation
- Language Detectors

# Application: Spam Detection

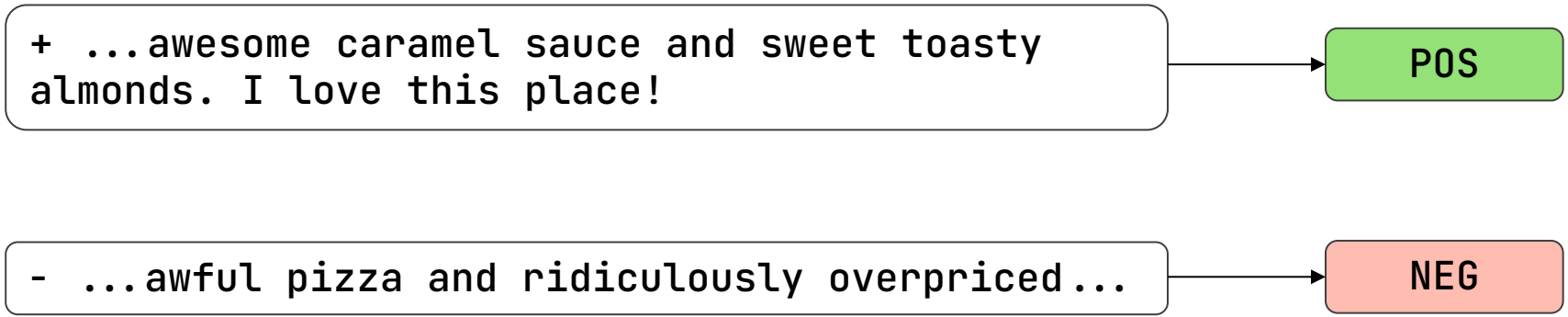
Binary classification: spam or not spam




Features that help:
sender address, keywords ("free", "winner"), formatting, links

# Application: Sentiment Analysis

Classify the polarity of a text: positive, negative, (neutral)

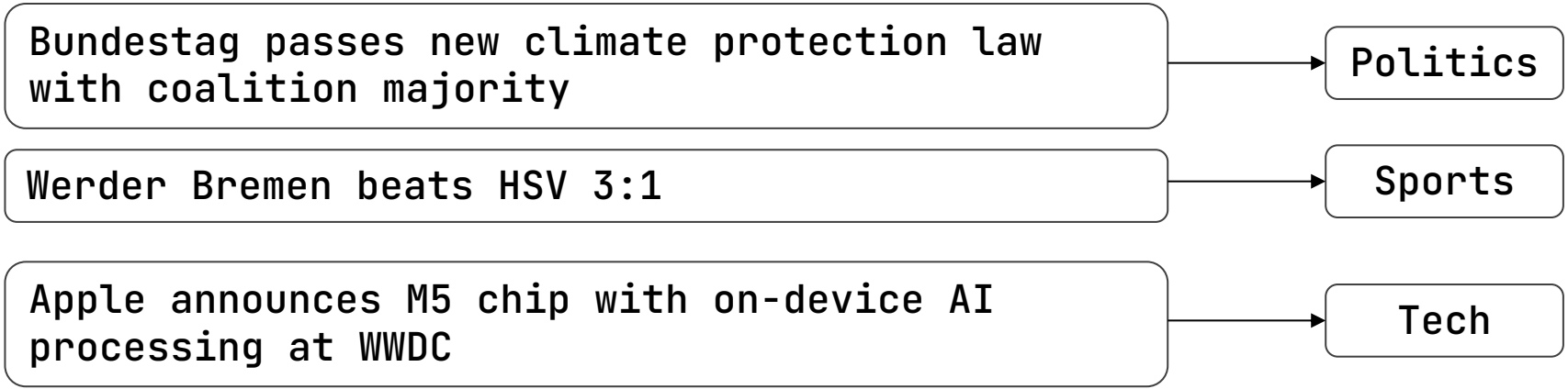



Used in:

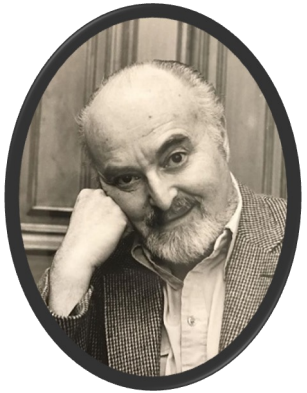
- product reviews, social media monitoring, political analysis, stock market prediction

# Application: Topic Classification

Assign documents to predefined categories

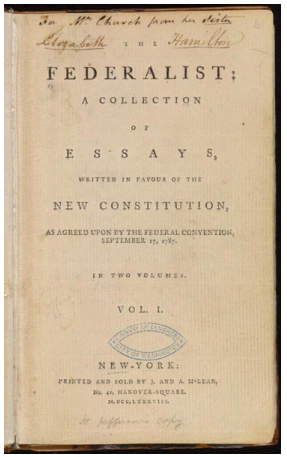


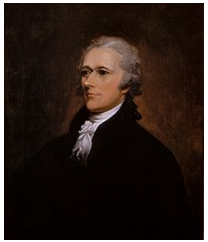

- Subject category classification is the task for which ***Naive Bayes*** was invented.
- Melvin Earl ("Bill") Maron (1961) at the RAND Corporation built the first NB classifier to assign subject categories to journal abstracts.



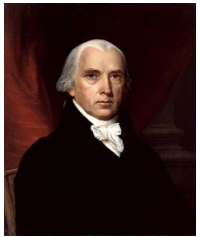
# Application: Who wrote which Federalist Papers?

- 85 anonymous essays written by Alexander Hamilton, James Madison, and John Jay to convince New York to ratify the U.S. Constitution.
- Problem: 12 of the papers had disputed authorship.
- Solution (Mosteller & Wallace, 1964): Used Bayesian analysis of function word frequencies ("an", "of", "upon", "by") to determine the author.
- Result: Madison wrote all 12 disputed papers.
- First applied Bayesian statistical analysis



Alexander Hamilton



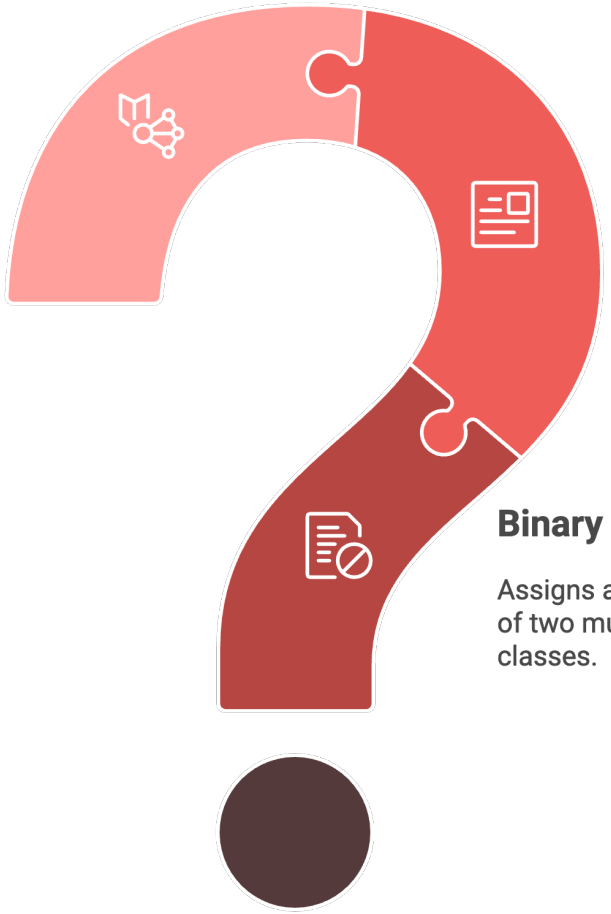
James Madison

Mosteller & Wallace (1964): Inference and Disputed Authorship: The Federalist

# Typology of Classification

**Multi-label Classification**

Assigns a document to a set of non-mutually exclusive classes.



**Multi-class Classification**

Assigns a document to one of several mutually exclusive classes.

**Binary Classification**

Assigns a document to one of two mutually exclusive classes.

# How Do We Build a Classifier?

1. ***Rule-based systems***

- Hand-crafted rules: `IF "free money" AND "lottery" → SPAM`
- Pro: interpretable, fast. Con: fragile, expensive to maintain.

2. ***Classical machine learning*** ← focus of this lecture

- Learn a mapping from feature vectors to labels using training data.
- Naive Bayes, Logistic Regression, SVM, ...

3. ***Deep Learning*** ← covered in later sessions

- Neural networks learn feature representations automatically.
- CNNs, RNNs, Transformers (BERT)

# Linear vs. Nonlinear Classifiers

## Linear Classifiers

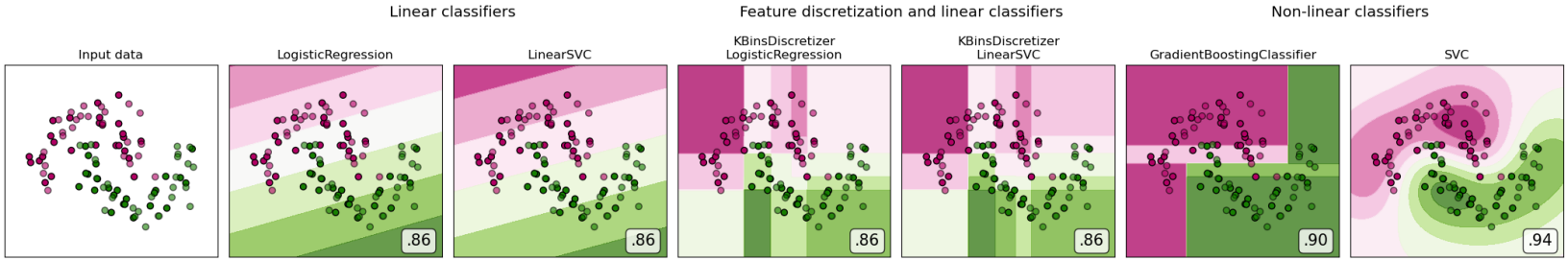
find a hyperplane (a straight line in 2D, a flat plane in 3D, ...) that separates classes in feature space.

→ Naive Bayes*, Logistic Regression, Linear SVM, Perceptron

## Nonlinear Classifiers

can learn curved, complex decision boundaries.

→ Decision Trees, Random Forests, Neural Networks, nonlinear Kernel SVM, BERT

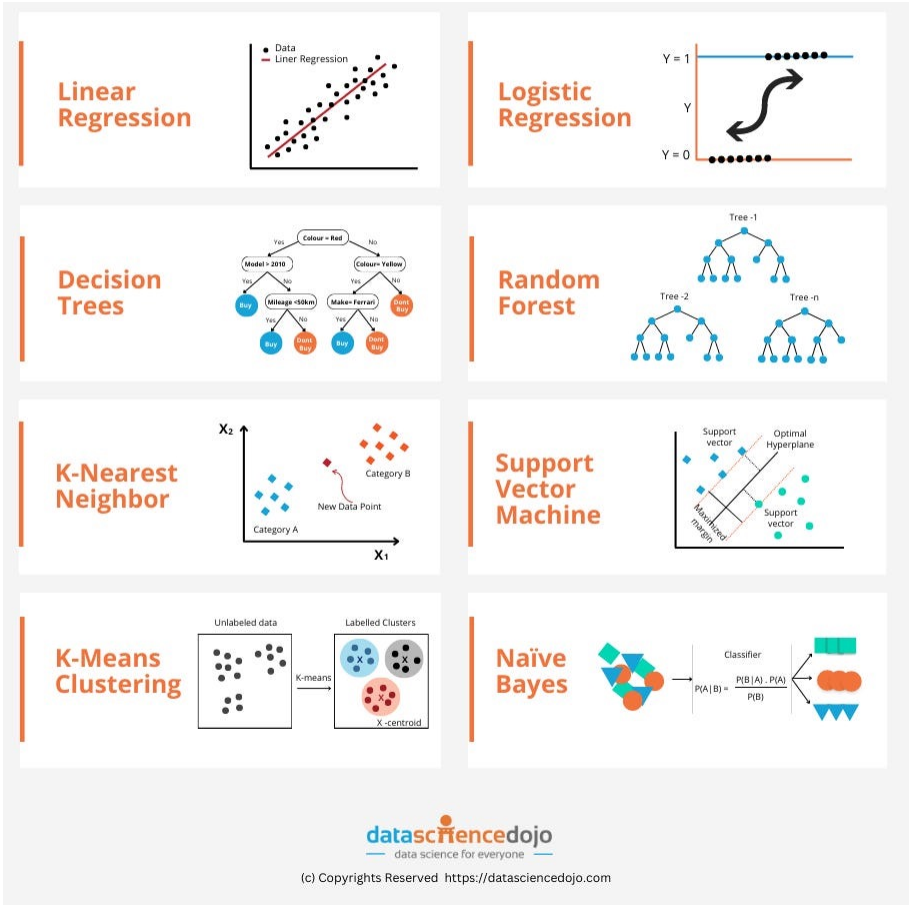



*Multinomial NB (the variant we use for text) is linear in log-probability space: the decision rule is a weighted sum of log $P(w_i|c)$ terms. Other NB variants like Gaussian NB can produce curved boundaries in the original feature space.

# Generative vs. Discriminative Classifiers

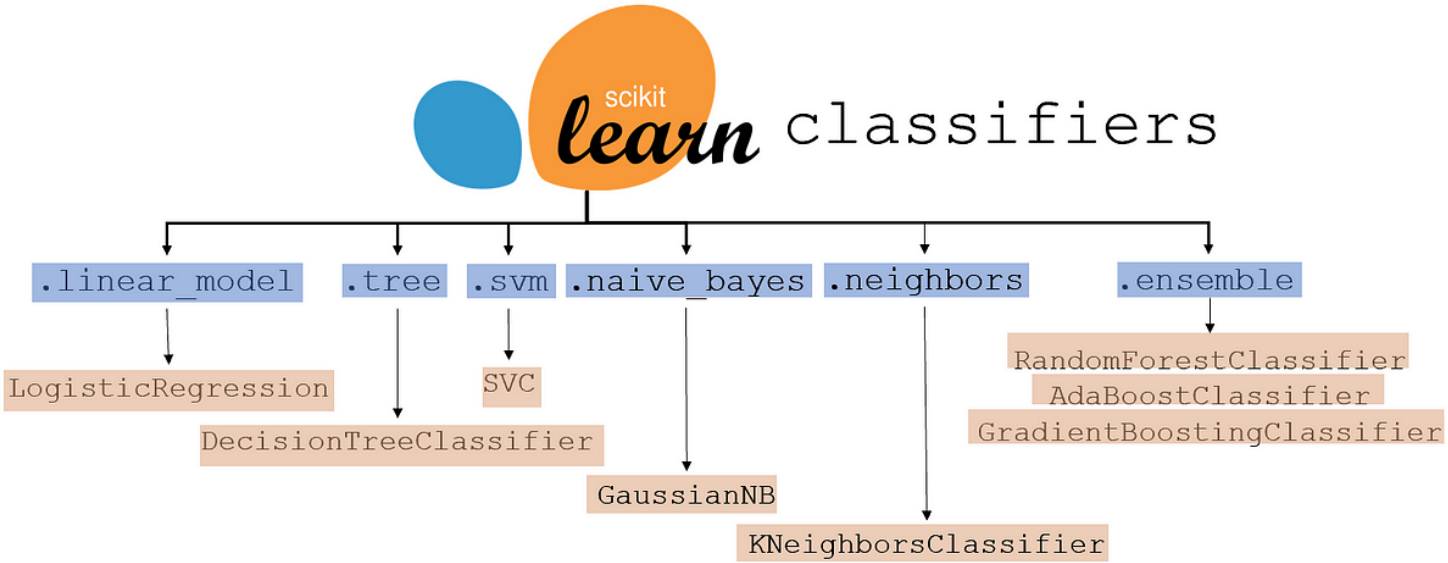
| | Generative (e.g., Naive Bayes) | Discriminative (e.g., LogReg) |
|---|---|---|
| **Philosophy** | Model how each class generates data | Model what distinguishes the classes |
| **Question** | "What does a positive review look like?" | "What features tell positive from negative?" |
| **Models** | $P(doc\|class) \times P(class)$, then Bayes' rule | $P(class\|doc)$ directly |
| **Correlated features** | Struggles (double-counts) | Handles well |
| **Small data** | Often better | Needs more data |
| **Training speed** | Very fast (just counting) | Iterative optimization |
| **In practice** | Good baseline, fast | Usually more accurate |

# Common Classifiers

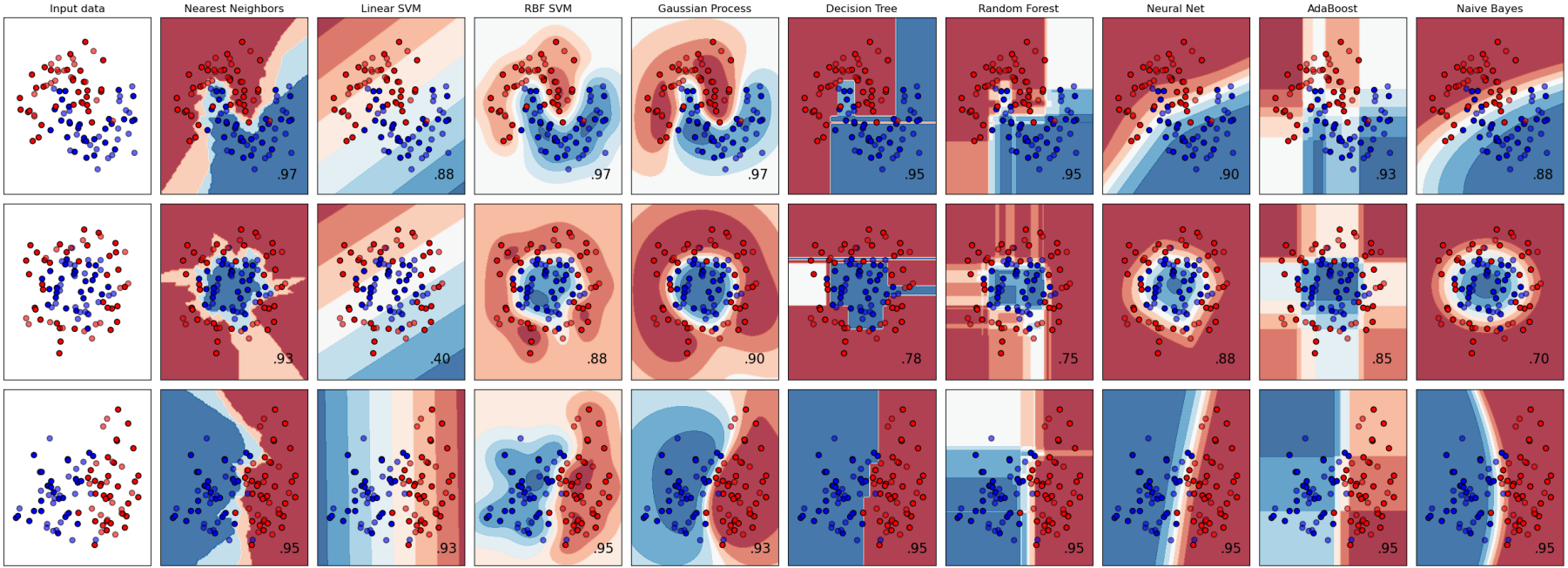



datasciencedojo
data science for everyone

# Common Classifiers



# Common Classifiers



01.

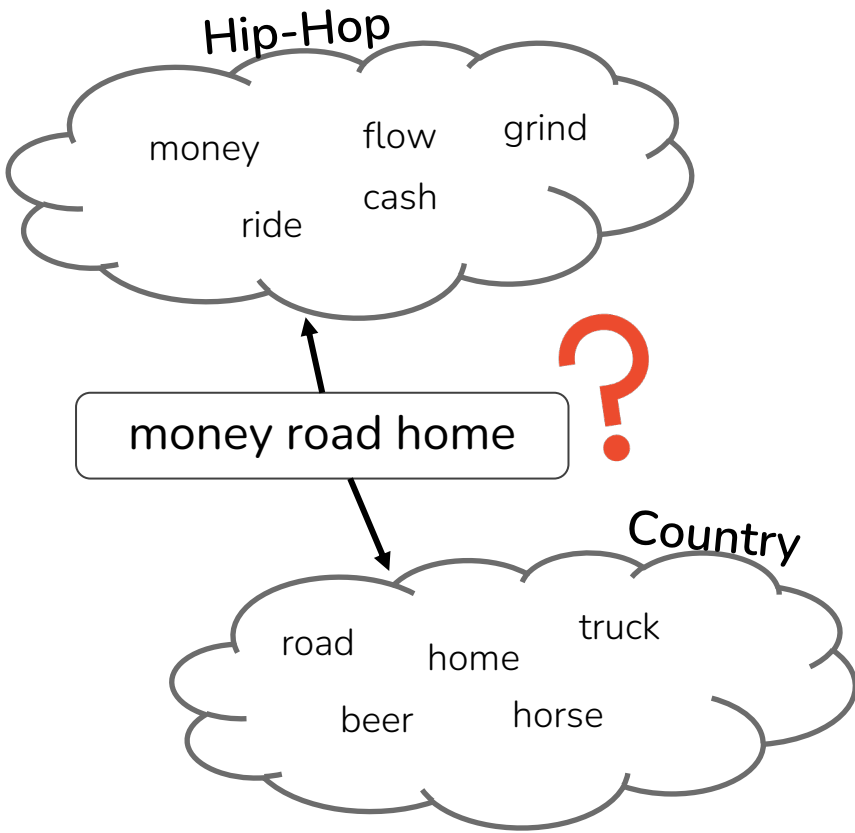
# Naïve Bayes

# Naive Bayes: The Idea

A simple probabilistic classifier based on ***Bayes' Theorem***.

Given a new document, ask: which class most likely generated this text?

- Count how often words appear in each class during training
- For a new document, multiply those word probabilities together
- Pick the class with the highest score




**Despite its simplifying assumptions, Naive Bayes is often surprisingly effective and remains a strong baseline for text classification.**

# Recall: Bag of Words

- For Naive Bayes, we treat each document as a bag of words: we care about which words appear and how often, but not about their order.
- Position doesn't matter: *"money in the bank"* and *"bank the money in"* are the same bag.

I got my mind on my money and my money on my mind

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| I | 1 | got | 1 | my | 4 | mind | 2 | on | 2 | money | 2 | and | 1 |

# Bayes' Theorem

***Given:*** a document $d = w_1, \ldots, w_k$ and a set of classes $C$

***Goal:*** find the class $c$ that is most probable given $d$

**Likelihood**
Probability of seeing $\boldsymbol{d}$ if it belongs to class $\boldsymbol{c}$

**Prior**
How common is class $\boldsymbol{c}$ overall?

$$P(c \mid d) = \frac{P(d \mid c)\,P(c)}{P(d)}$$

**Posterior**
Probability of class $\boldsymbol{c}$ given document $\boldsymbol{d}$

**Evidence**
Total probability of $\boldsymbol{d}$ (same for all classes)

# MAP Estimation

- MAP = Maximum A Posteriori (“maximum after seeing the evidence”)
- **Goal (MAP):**

$$\hat{c}_{\text{MAP}} = \arg\max_{c \in C} P(c \mid d)$$

- **Apply Bayes’ Theorem:**

$$\hat{c}_{\text{MAP}} = \arg\max_{c \in C} \frac{P(d \mid c)\, P(c)}{P(d)}$$

- Since $P(d)$ is the same for all classes, we can drop it:

$$\hat{c}_{\text{MAP}} = \arg\max_{c \in C} P(d \mid c)\, P(c)$$

# The "Naive" Assumption

- ***Problem:*** $P(d|c) = P(w_1, w_2, \ldots, w_k \mid c)$ has too many parameters to estimate.
  - For a vocabulary of 10,000 words and a document of 100 words, we'd need to estimate probabilities over all possible 100-word combinations. Impossible.
- ***Solution:*** Assume every word is conditionally independent of every other word, given the class:

$$P(w_1, w_2, \ldots, w_k| \, c) = P(w_1|c) \cdot P(w_2|c) \cdot \ldots \cdot P(w_k|c) = \prod_{i=1}^{k} P(w_i|c)$$

This is clearly wrong (e.g., $P(\text{"Long"} \mid \textit{lyric contains "Beach"})$ is not independent in Snoop Dogg songs) but it works well in practice.

# The Full Naive Bayes Classifier

$$c_{NB} = \arg\max_{c \in C} P(c) \prod_{i=1}^{k} P(w_i|c)$$

1. For each class, multiply:

- Prior $P(c)$: How common is this class in the training data?
- Likelihoods $P(w_i|c)$: For each word in the document, how likely is it under this class?

2. Pick the class with the highest product.

**How do we estimate the likelihoods?**

# Estimating Probabilities from Training Data

- ***Prior:*** Count how many documents belong to each class.

$$\hat{P}(c_j) = \frac{\text{Count}(c_j)}{N_{\text{documents}}}$$

- ***Likelihood:*** For each word, count how often it appears in documents of that class, divided by the total word count for that class.

$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j)}{\sum_{w \in V} \text{Count}(w, c_j)}$$

How often that word appears across the whole class.

# Example: Genre Classification

money cash money hustle grind money

cash flow money ride hustle

road truck beer road home

home beer truck road horse

Priors: $P(\text{Hip-Hop}) = 0.5$

$P(\text{Country}) = 0.5$

money road home ?

| Word | Hip-Hop | Country |
|---|---|---|
| **money** | 4 | 0 |
| **cash** | 2 | 0 |
| **hustle** | 2 | 0 |
| **grind** | 1 | 0 |
| **flow** | 1 | 0 |
| **ride** | 1 | 0 |
| **road** | 0 | 3 |
| **truck** | 0 | 2 |
| **beer** | 0 | 2 |
| **home** | 0 | 2 |
| **horse** | 0 | 1 |
| **Total** | **11** | **10** |

"money" never appears in Country → zeros out entire product

| Word | P(w \| Hip-Hop) | P(w \| Country) |
|---|---|---|
| **money** | 4/11 = 0.364 | 0/10 = **0.000** |
| **road** | 0/11 = 0.000 | 3/10 = 0.300 |
| **home** | 0/11 = 0.000 | 2/10 = 0.200 |

# Laplace (Add-α) Smoothing

- If any word in the test document has count 0 for a class, the entire product becomes 0

$$P(c) \cdot \prod P(w_i|c) = P(c) \cdot \ldots \cdot \underbrace{0}_{P(money|\text{Country})} \cdot \ldots = 0$$

- We need a way to assign small but nonzero probabilities to unseen words.
- Idea: Add a small count α to every word in every class. Pretend we have seen every word at least α times.

$$\hat{P}(w_i|c_j) = \frac{\text{Count}(w_i, c_j) + \alpha}{\sum_{w \in V} \text{Count}(w, c_j) + \alpha|V|}$$

- α = 1 is called Laplace smoothing (add-1)
- α can be tuned as a hyperparameter (in sklearn: `MultinomialNB(alpha=...)`)

# Continue Example

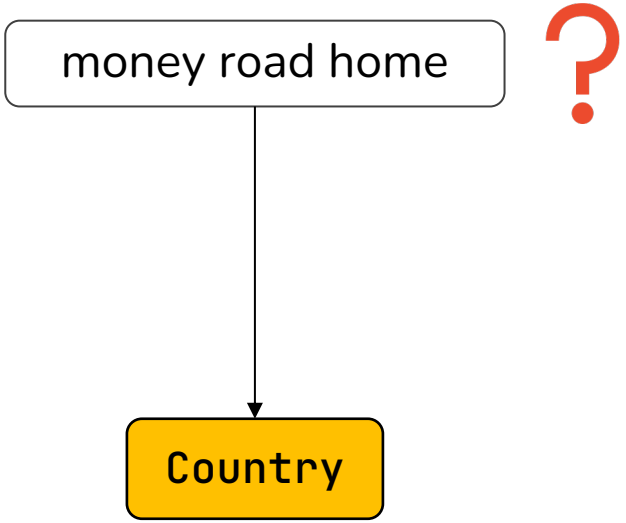
| Word | P(w \| Hip-Hop) | P(w \| Country) |
|---|---|---|
| **money** | (4+1)/22 = 5/22 ≈ 0.227 | (0+1)/21 = 1/21 ≈ 0.048 |
| **road** | (0+1)/22 = 1/22 ≈ 0.045 | (3+1)/21 = 4/21 ≈ 0.190 |
| **home** | (0+1)/22 = 1/22 ≈ 0.045 | (2+1)/21 = 3/21 ≈ 0.143 |

Score each class

$$\text{Score}(c) = P(c) \cdot \prod_{w \in \text{doc}} P(w|c)$$

Hip-Hop: 0.5 × 0.227 × 0.045 × 0.045 = 0.5 × 0.000460 ≈ 0.000230

Country: 0.5 × 0.048 × 0.190 × 0.143 = 0.5 × 0.001304 ≈ 0.000652



# Log Space: Avoiding Underflow

- In practice, multiplying many small probabilities leads to floating-point underflow (numbers too small for the computer to represent).
- ***Solution:*** work in log space. Since log is monotonic, it preserves the ranking:

$$c_{NB} = \arg\max_{c \in C} \left[ \log P(c) + \sum_{i \in \text{words}} \log P(w_i|c) \right]$$

Naive Bayes is a linear classifier (in log space): the decision is a weighted sum of features.

# When to use Naïve Bayes

| Strengths | Weaknesses |
|---|---|
| Very fast to train (just counting) | Independence assumption is too strong |
| Low storage requirements | Doesn't model feature interactions |
| Works well with small training data | Struggles with highly imbalanced classes |
| Robust to irrelevant features (they cancel out) | Probability estimates can be poorly calibrated |

**When to use NB:**

- You need a quick baseline
- Training data is limited
- You want interpretable word-level probabilities per class

**When to look elsewhere:**

- Features are correlated (e.g., n-grams, rich feature sets)
- You need well-calibrated probabilities
- You have enough data for a discriminative model

# Naïve Bayes Can Use Any Features

Naive Bayes is not limited to words. You can use any set of features:

- URLs, email addresses, sender domain
- Capitalization patterns, punctuation
- Character n-grams (useful for language identification)
- Domain-specific features (e.g., song duration for genre classification)

$$P(d|c) = P(f_1|c) \cdot P(f_2|c) \cdot \ldots \cdot P(f_K|c)$$

02.

# Perceptron

# From Generative to Discriminative

Naive Bayes asks: *"How does each class generate text?"*

But what if we instead ask: *"What features best distinguish the classes?"*

This is the ***discriminative*** approach. Instead of modeling $P(d|c)$ for each class, we directly learn $P(c|d)$.

To get there, we'll take three steps:

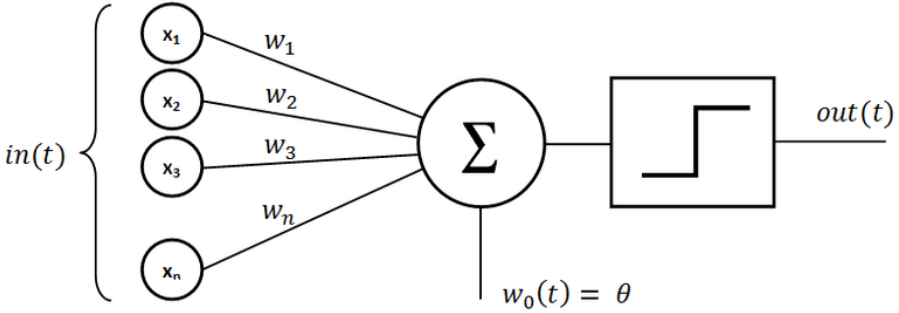
1. Perceptron: The simplest discriminative classifier (hard decisions)
2. Logistic Regression: Add probabilities (sigmoid + loss function)
3. Neural Networks: Stack them for nonlinear power

# The Perceptron: Simplest Discriminative Classifier

**Input:** Feature vector $x = (x_1, x_2, \ldots, x_n)$

**Weights:** $w = (w_1, w_2, \ldots, w_n)$ + bias $b$

**Output:** $\hat{y} = sign(w \cdot x + b)$



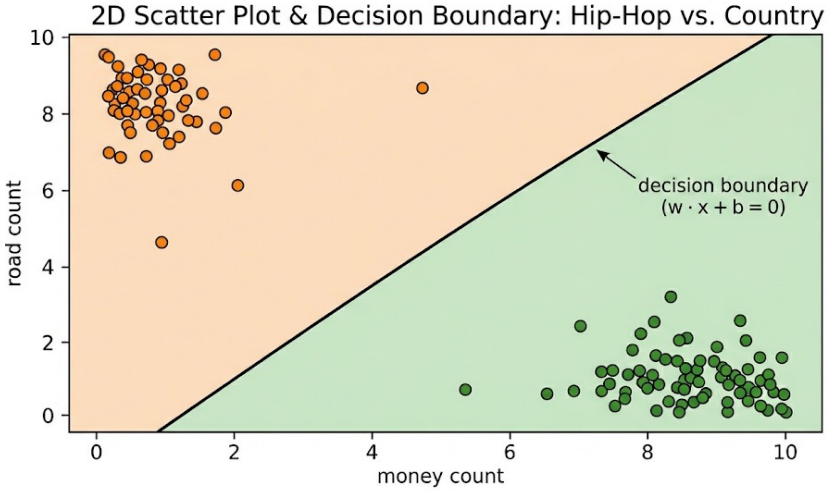

Simplest Neural Network

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w} \cdot \mathbf{x} + b$$

$$\hat{y} = \begin{cases} +1 \text{ if } z > 0 \\ -1 \text{ if } z \leq 0 \end{cases}$$

The decision boundary is the hyperplane where $w \cdot x + b = 0$.

No probabilities. Just a hard yes/no decision.



# How the Perceptron Learns

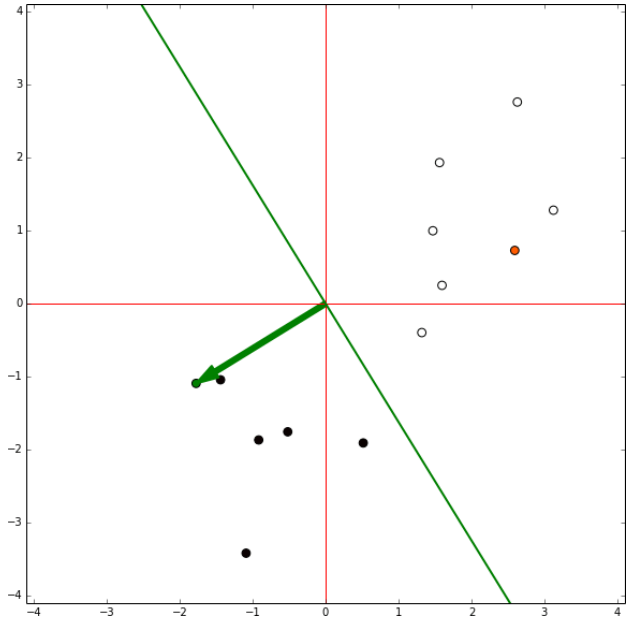
If the prediction is correct: do nothing.

If the prediction is wrong: nudge the weights toward the correct answer:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (y - \hat{y}) \cdot \mathbf{x}$$

where $\eta$ is the learning rate.

- **Error-driven:** Only learns when it makes a mistake
- **Converges** if the data is linearly separable (Perceptron Convergence Theorem)
- Cannot learn **non-linearly-separable** problems (XOR problem)



03.

# Logistic Regression

# From Perceptron to Logistic Regression

The perceptron gives a hard decision: $sign(w \cdot x + b) \rightarrow +1$ or $-1$.

But we often want to know how confident the model is.

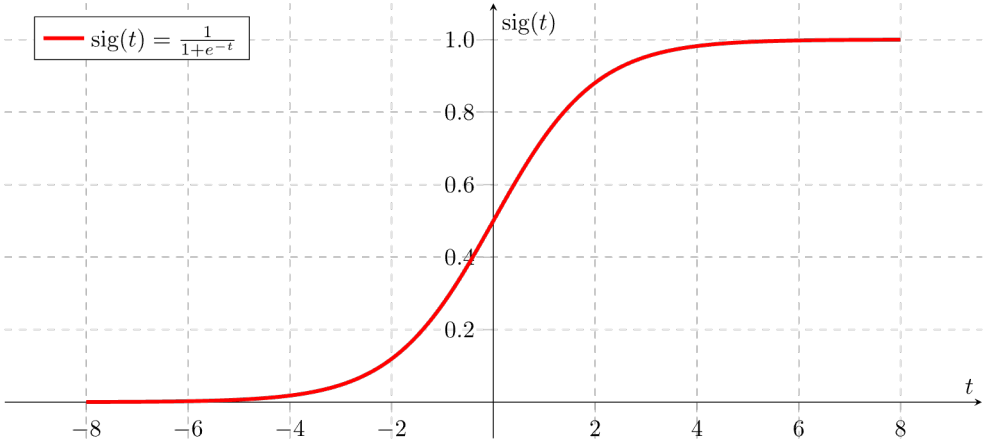
*"Is this lyric 51% Hip-Hop or 99% Hip-Hop?"*

Idea: Replace the hard sign function with a smooth function that outputs a probability:

| **Perceptron** | **Logistic Regression** |
|---|---|
| $\hat{y} = \text{sign}(w \cdot x + b)$ | $\hat{y} = \sigma(w \cdot x + b)$ |
| Output: -1 or +1 | Output: probability in (0, 1) |
| Hard decision | Soft, graded decision |

# The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$




- Maps any real number to the range (0, 1)
- $\sigma(0) = 0.5$ (the decision boundary)
- Decision rule: predict class 1 if $P(y = 1|x) > 0.5$, else class 0.
- Large positive z $\rightarrow$ $\sigma(z) \approx 1$ (confident positive)
- Large negative z $\rightarrow$ $\sigma(z) \approx 0$ (confident negative)
- Nearly linear around 0, squashes outliers toward 0 or 1

# Example

- Task: Hip-Hop (1) vs. Country (0)
- Test lyric: "riding down the old dirt road with the bass bumping loud, cash in my pocket"

$$z = (3.0\times2) + (-3.0\times2) + (2.0\times0) + (2.5\times1) + (-2.0\times1) + (0.5\times2.64) + 0.1 = 1.92$$

$$P(\text{Hip-Hop}|\mathbf{x}) = \sigma(1.92) = \frac{1}{1+e^{-1.92}} \approx 0.87$$

| Feature | Description | Weight | Value |
|---|---|---|---|
| $\mathbf{x_1}$ | Count of Hip-Hop vocab hits (cash, grind, hustle, ...) | +3.0 | 2 |
| $\mathbf{x_2}$ | Count of Country vocab hits (road, truck, dirt, ...) | -3.0 | 2 |
| $\mathbf{x_3}$ | Contains slang markers (yo, aight, finna) (0/1) | +2.0 | 0 |
| $\mathbf{x_4}$ | Mentions "bass", "beat", or "flow" (0/1) | +2.5 | 1 |
| $\mathbf{x_5}$ | Mentions "truck", "dog", or "mama" (0/1) | -2.0 | 1 |
| $\mathbf{x_6}$ | log(word count) | +0.5 | 2.64 |

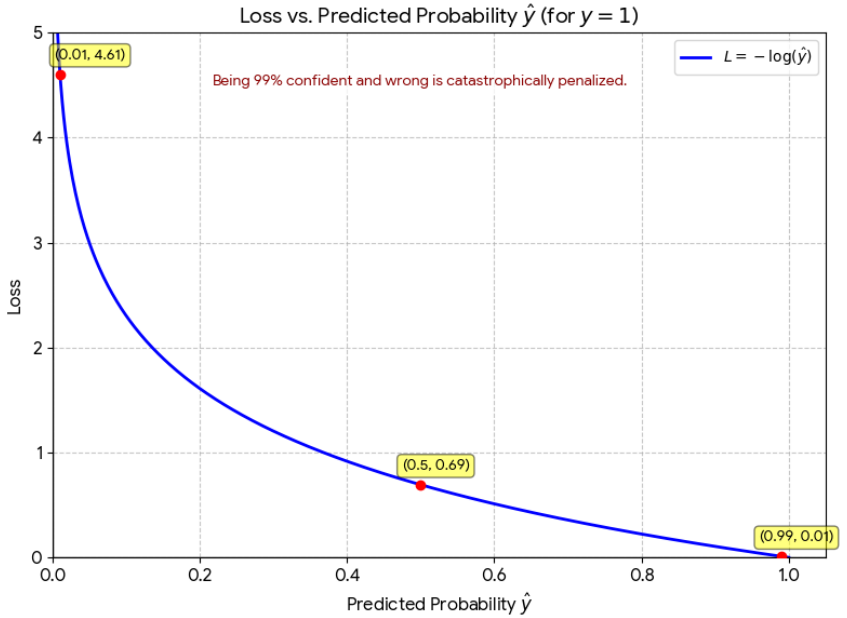
The model is 87% confident this lyric is Hip-Hop.
Since 0.87 > 0.5 → predict Hip-Hop.

# How Does Logistic Regression Learn?

***Cross-entropy loss:***

$$L = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

- True label y = 1, model predicts ŷ = 0.99 → loss ≈ 0.01 (very small, good!)
- True label y = 1, model predicts ŷ = 0.51 → loss ≈ 0.67 (mediocre)
- True label y = 1, model predicts ŷ = 0.01 → loss ≈ 4.61 (catastrophic!)
- Training is done by ***gradient descent*** (not covered in this class)



# Why Log Probabilities? (Again)

**Problem 1:** Underflow. Multiplying many small probabilities → numbers get too small quickly

**Problem 2:** Optimization. Sums are easier to differentiate and optimize than products.

Log transforms:

- Products → sums
- Division → subtraction
- Exponents → multiplication

This is why cross-entropy uses log(ŷ), why NB works in log-space, and why neural networks use log-softmax.

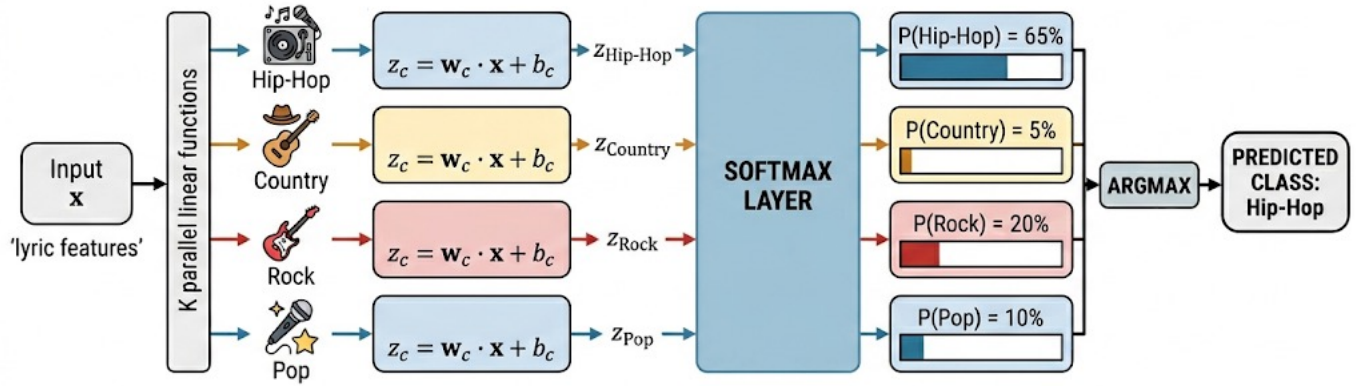
# Multinomial Logistic Regression (Softmax)

Binary logistic regression handles 2 classes. For K > 2 classes (e.g., genre classification with Hip-Hop, Country, Rock, Pop) we need:

Softmax function:

$$P(y = c \mid x) = \frac{e^{\mathbf{w_c} \cdot \mathbf{x} + b_c}}{\sum_{j=1}^{K} e^{\mathbf{w_j} \cdot \mathbf{x} + b_j}}$$

Each class gets its own weight vector $w_c$ and bias $b_c$.



# Regularization

With many features (e.g., 10,000 TF-IDF terms), logistic regression can overfit: it memorizes the training data instead of learning general patterns.

Regularization adds a penalty for large weights:

$$\hat{\theta} = \arg\min \left[ \text{Loss}(\theta) + \lambda \sum_j \theta_j^2 \right]$$

This discourages the model from relying too heavily on any single feature.

- $\lambda$ (or C = 1/$\lambda$ in sklearn) controls the strength of regularization (1 by deault)
- Small C = strong regularization (simpler model)
- Large C = weak regularization (more flexible model)

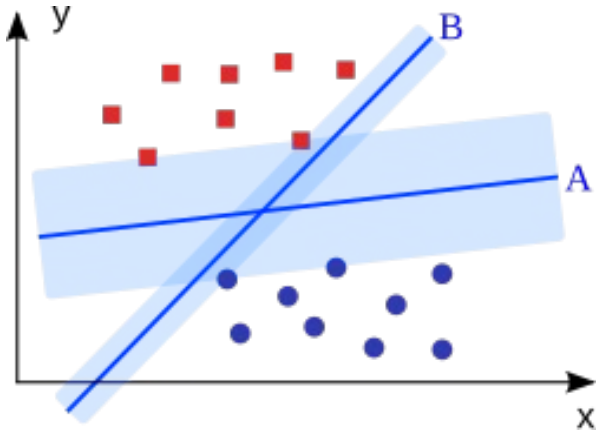
04.

# Other Classifiers

# Support Vector Machines (SVMs)

Like logistic regression, SVMs find a linear decision boundary. But the optimization objective is different:

SVM idea: Find the hyperplane that maximizes the margin between the closest points of different classes.




The closest points to the boundary are called support vectors. Only these matter for defining the boundary; all other points are irrelevant.

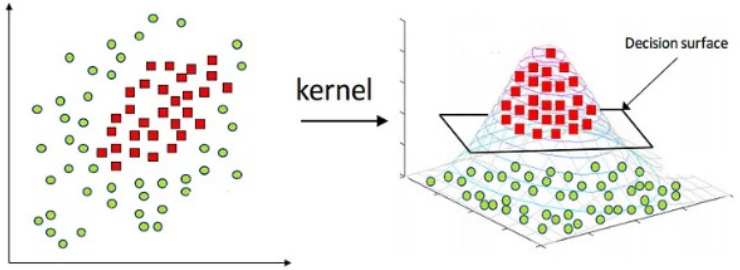
Why SVMs work well for text:

- High-dimensional sparse feature spaces (like TF-IDF) tend to be linearly separable
- SVMs are designed to generalize well even with many features and few training examples

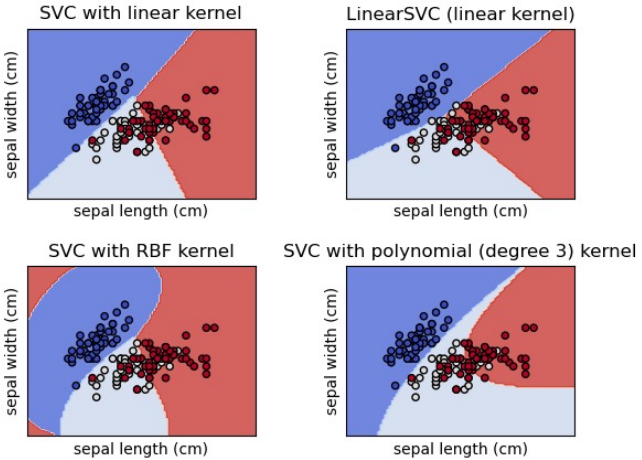
# SVM Kernels

What if the data is not linearly separable?

The kernel trick: Transform the data into a higher-dimensional space where a linear boundary works, without actually computing the transformation.




| Kernel | Decision Boundary | When to Use |
|---|---|---|
| **Linear** | Straight line / hyperplane | Text classification (high-dim sparse data) |
| **RBF (Radial Basis Function)** | Curved, circular regions | Low-dimensional continuous features |
| **Polynomial** | Curved (degree-dependent) | Captures combinations of features (e.g., word pairs) |



# Tree-Based Methods

Tree-based methods are strong on tabular data but usually underperform linear classifiers on text with BoW/TF-IDF features

## *Decision Trees*

- Recursively split data based on features
- Internal nodes = feature tests, leaf nodes = class labels
- Pro: Highly interpretable
- Con: Prone to overfitting, poor on high-dimensional sparse text data

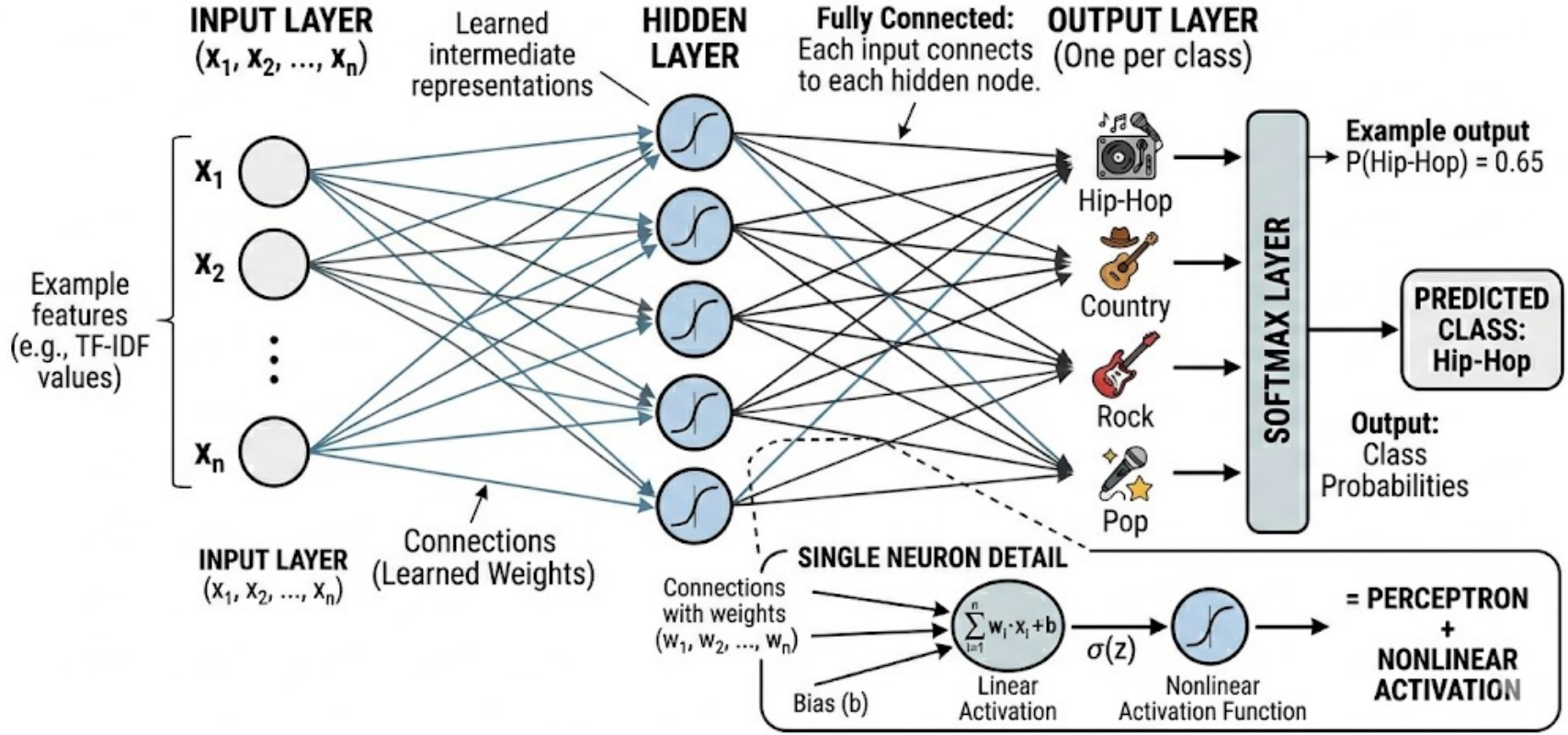
## *Random Forests*

- Ensemble of many decision trees, each trained on a random subset of data and features
- Majority vote across all trees
- More robust than a single tree, handles overfitting better
- Nonlinear: can learn complex decision boundaries

# Neural Networks: A Preview

- A single perceptron can only learn linear boundaries. What if we stack them?
- ***Neural Network:*** multiple layers of perceptrons with nonlinear activation functions between them.
- ***Input layer:*** Your features (e.g., TF-IDF vector)
- ***Hidden layer(s):*** Learned intermediate representations
- ***Output layer:*** Class probabilities (via softmax)
- With one hidden layer and enough neurons, a neural network can approximate any continuous function (Universal Approximation Theorem).
- This is how we go from linear (NB, LogReg, SVM) to nonlinear.
- A single logistic regression unit is a neural network with zero hidden layers.

# SIMPLE NEURAL NETWORK ARCHITECTURE



05.

# Evaluation

# Why Evaluate?

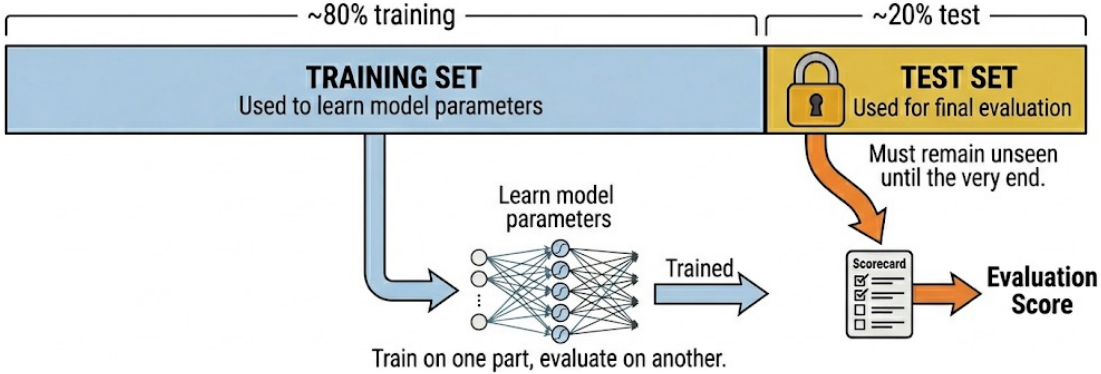
A classifier that looks good on training data might fail on new, unseen text.

Evaluation tells us:

- How well does the model generalize? (performance on unseen data)
- Which model is better? (comparing algorithms or hyperparameter settings)
- Where does the model fail? (diagnosing weaknesses, guiding improvement)
- Is this model good enough for deployment?

# Data Partitioning

- To get an honest estimate of performance, we split our data:
- Training set (~80%): Used to learn model parameters.
- Test set (~20%): Used for final evaluation. Must remain unseen until the very end.
- For hyperparameter tuning (e.g., α in NB, C in LogReg), you can further split training into train + validation/dev set, or use cross-validation.
- Stratified splits: When classes are imbalanced, use **`stratify=y`** in **`train_test_split`** to ensure each split has the same class proportions as the full dataset.




Never evaluate on data the model has seen during training.
That measures memorization, not generalization

# The Confusion Matrix

For binary classification (e.g., Hip-Hop vs. Country), the confusion matrix shows all four possible outcomes of a prediction:

**True Positive (TP)**

Correctly identified as Hip-Hop

**False Positive (FP)**

Country song misclassified as Hip-Hop

**False Negative (FN)**

Hip-Hop song misclassified as Country

**True Negative (TN)**

Correctly identified as Country

# Metrics

**True Positive (TP)**

Correctly identified as Hip-Hop

**False Positive (FP)**

Country song misclassified as Hip-Hop

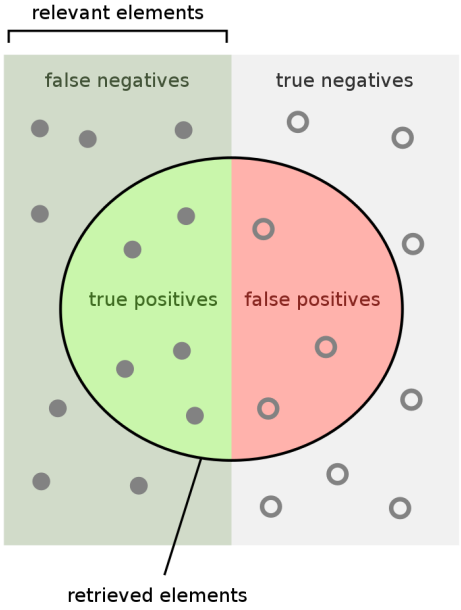
**False Negative (FN)**

Hip-Hop song misclassified as Country

**True Negative (TN)**

Correctly identified as Country

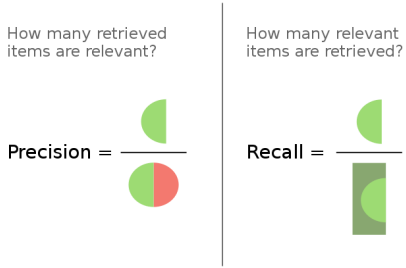
$$Precision = \frac{TP}{TP + FP}$$




$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$




*What fraction of all predictions were correct?*

# The Precision-Recall Trade-off

You usually can't maximize both at the same time.

- Most classifiers output a probability (e.g., P(Hip-Hop) = 0.87). We choose a threshold to make the final decision (default: 0.5).
- Lower the threshold (e.g., 0.3): "Call it Hip-Hop even if I'm only 30% sure"

→ More positive predictions → Recall goes up (catch more Hip-Hop) but Precision goes down (more false alarms)

- Raise the threshold (e.g., 0.8): "Only call it Hip-Hop if I'm very sure"

→ Fewer positive predictions → Precision goes up but Recall goes down (miss borderline cases)

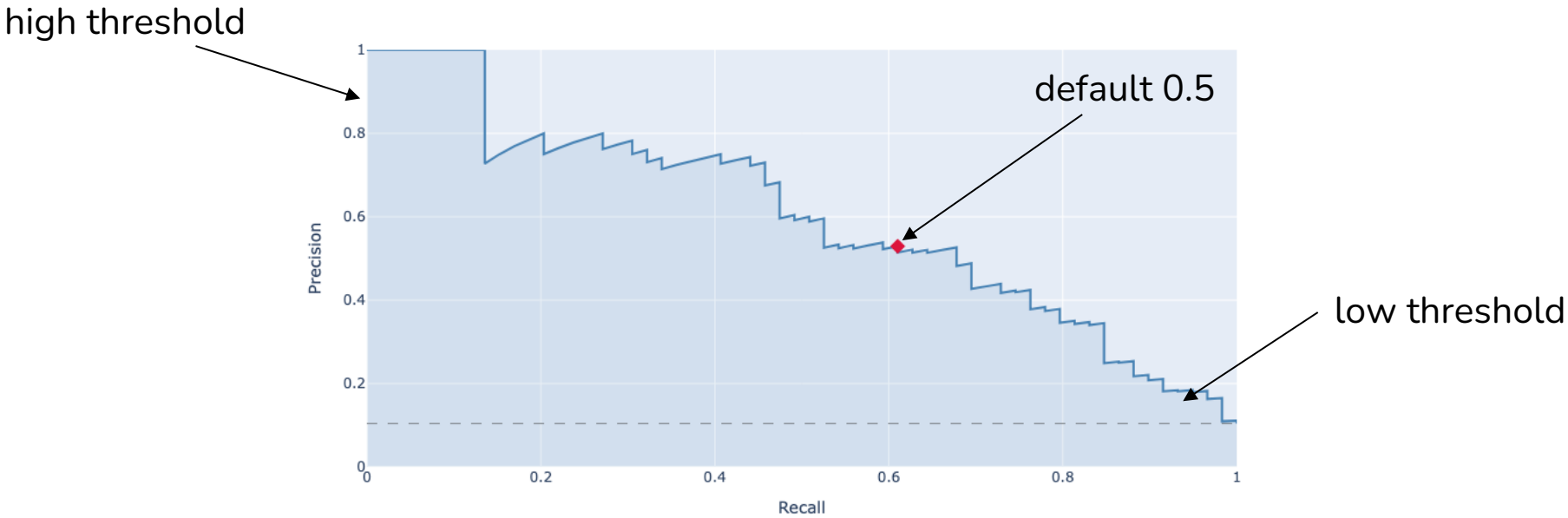
- The right trade-off depends on the application. There is no universal "best."

**It depends on the task** 🤷

# The Precision-Recall Trade-off

You can visualize this trade-off with a ***Precision-Recall curve:***

- plot precision vs. recall at various thresholds.
- The area under this curve (AUC-PR) gives a single summary score.



# F1-Score

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The harmonic mean of precision and recall.

Why harmonic mean? It punishes extreme imbalances. If either precision or recall is very low, F1 will be low too.

| Precision | Recall | F1 |
|---|---|---|
| 0.90 | 0.90 | 0.90 |
| 0.95 | 0.50 | 0.65 |
| 1.00 | 0.10 | 0.18 |

# Multi-Class Evaluation

- With more than 2 classes (e.g., Hip-Hop, Country, Rock), we compute precision, recall, and F1 per class by treating each class as a "one vs. rest" binary problem.
- For Hip-Hop: TP = correctly predicted Hip-Hop, FP = other genres predicted as Hip-Hop, FN = Hip-Hop predicted as something else.
- Same logic for Country, Rock, etc.
- This gives us a table of per-class metrics. But we often want a single number to summarize overall performance, so we take averages (next slide)

| | **Pred: Hip-Hop** | **Pred: Country** | **Pred: Rock** |
|---|---|---|---|
| **Actual: Hip-Hop** | 18 | 1 | 1 |
| **Actual: Country** | 2 | 8 | 0 |
| **Actual: Rock** | 10 | 1 | 59 |

| **Genre** | **Precision** | **Recall** | **F1** | **Support** |
|---|---|---|---|---|
| **Hip-Hop** | 18/30 = 0.60 | 18/20 = 0.90 | 0.72 | 20 |
| **Country** | 8/10 = 0.80 | 8/10 = 0.80 | 0.80 | 10 |
| **Rock** | 59/60 = 0.98 | 59/70 = 0.84 | 0.91 | 70 |

# Macro vs. Weighted Averaging

## *Macro averaging:*

Compute the metric for each class, then take the unweighted mean.

$$F_{1,\text{macro}} = \frac{F_{1,\text{HipHop}} + F_{1,\text{Country}} + F_{1,\text{Rock}}}{3} = \frac{0.72 + 0.80 + 0.91}{3} = 0.81$$

Every class counts equally, regardless of size. A rare genre matters as much as a common one.

## *Weighted averaging:*

Same as macro, but weight each class by its support (number of test instances).

$$F_{1,\text{weighted}} = \frac{n_{\text{HH}} \cdot F_{1,\text{HH}} + n_{\text{C}} \cdot F_{1,\text{C}} + n_{\text{R}} \cdot F_{1,\text{R}}}{n_{\text{total}}} = \frac{20{\times}0.72 + 10{\times}0.80 + 70{\times}0.91}{100} = 0.86$$

Larger classes contribute more. Reflects overall instance-level performance.

# The Imbalanced Data Problem

Many real-world datasets are heavily skewed:

- 95% of emails are not spam
- 99% of transactions are not fraud
- In our lyrics dataset, some genres have thousands of songs, others have only a few

Why this is a problem:

- A model can achieve high accuracy by always predicting the majority class
- The model never learns to recognize the minority class
- Standard loss functions treat all errors equally, so the majority class dominates training

# Strategies for Imbalanced Data

- **Use the right metric.**
  Report F1 (especially macro F1) instead of accuracy. Accuracy hides the problem; macro F1 exposes it.
- **Stratified splits.**
  Use `stratify=y` when splitting data to preserve class proportions in both train and test sets.
- **Class weights.**
  Most sklearn classifiers support `class_weight='balanced'`, which automatically upweights minority classes in the loss function. The model gets penalized more for misclassifying a rare genre.
- **Filter rare classes.**
  For very small classes, there may not be enough data to learn from. Sometimes the pragmatic choice is to exclude them.

# Cross-Validation for Robust Evaluation

- A single train/test split might produce results sensitive to the specific partitioning of data
- K-Fold Cross-Validation:
  - Partition the dataset into K non-overlapping subsets (folds) of equal size
  - For each k from 1 to K:
    - Use fold k as the test set
    - Use the remaining K-1 folds as the training set
    - Train the model and compute the evaluation metric(s) on fold k
- The overall performance is typically the average of the K metrics obtained
- Provides a more robust estimate of the model's generalization performance and reduces variance of the estimate